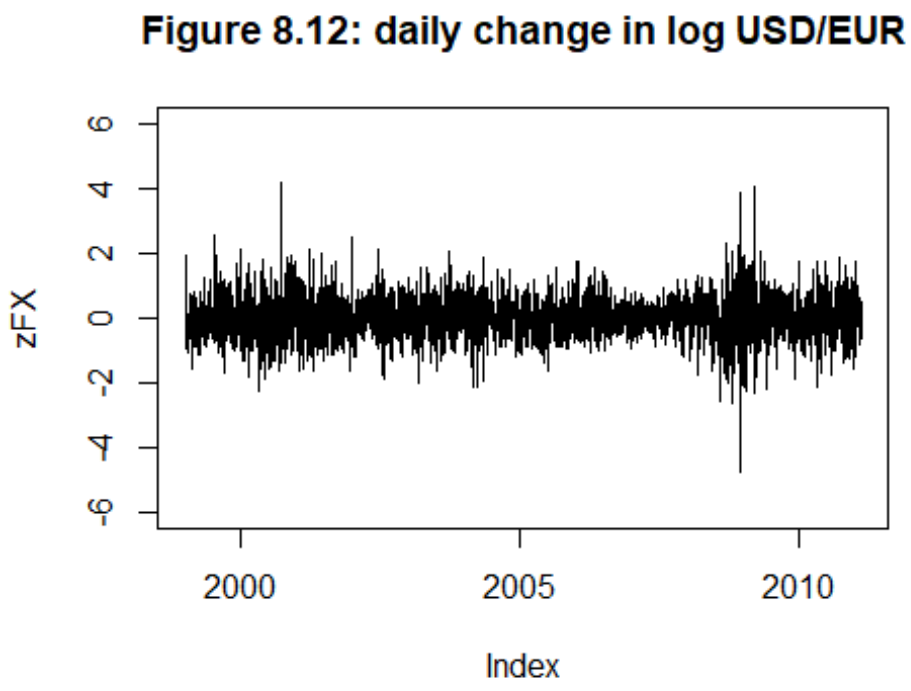
GML_04_03-usdeur.R

frank

2024-05-06

```
# feb 2023
# r script for simulation of GARCH
# written by Frank Lehrbass (->lehrbass.de) for teaching purposes only
# students are solely responsible for whatever they do with this coding
# this R coding is WITHOUT ANY WARRANTY

#is an example from Verbeek

# Step 1

rm(list=ls(all=TRUE))

# Load packages

library(denstrip)
library(tseries)

## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo

library(fGarch)

## NOTE: Packages 'fBasics', 'timeDate', and 'timeSeries' are no longer
## attached to the search() path when 'fGarch' is attached.
##
## If needed attach them yourself in your R script by e.g.,
##         require("timeSeries")

library(zoo)

##
## Attache Paket: 'zoo'

## Die folgenden Objekte sind maskiert von 'package:base':
##
##     as.Date, as.Date.numeric

library(rugarch)

## Lade nötiges Paket: parallel

##
## Attache Paket: 'rugarch'
```

```
## Das folgende Objekt ist maskiert 'package:stats':
##
##     sigma

library(moments)

data_table <- read.table("GML_04_03 usdeur.csv", header=TRUE,
sep=";",dec=",")

usd  <- data_table$usd
dates <- data_table$Date

DateStamp = as.Date(dates,format="%d.%m.%Y")
zusd = zoo(usd,DateStamp)

zFX <- na.omit(zusd)
FX = coredata(zFX)

x11()
#Figure 8.12
plot(zFX, ylim = c(-6,6), main = "Figure 8.12: daily change in log USD/EUR")
```




```
length(zFX)

## [1] 3087

summary(FX)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -4.735441 -0.380585  0.008172  0.004693  0.393003  4.204134
```

```
kurtosis(FX)
```

```
## [1] 5.591918
```

```
skewness(FX)
```

```
## [1] 0.1038479
```

```
set.seed(123)
skewness(rnorm(1000,0,1))#zum Vgl
```

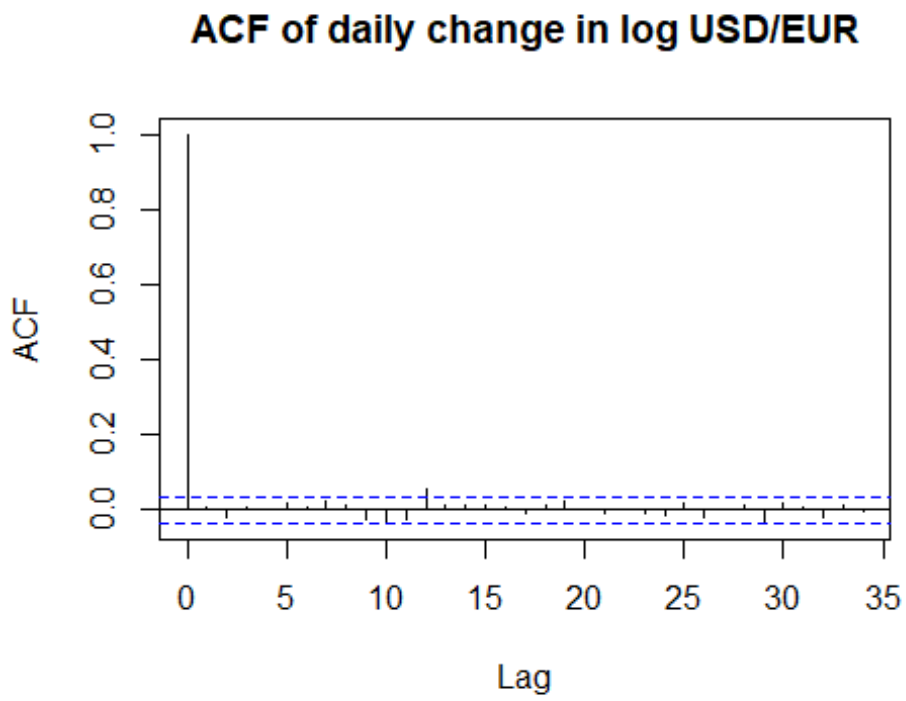
```
## [1] 0.06529391
```

```
adf.test(zFX)#series is stationary
```

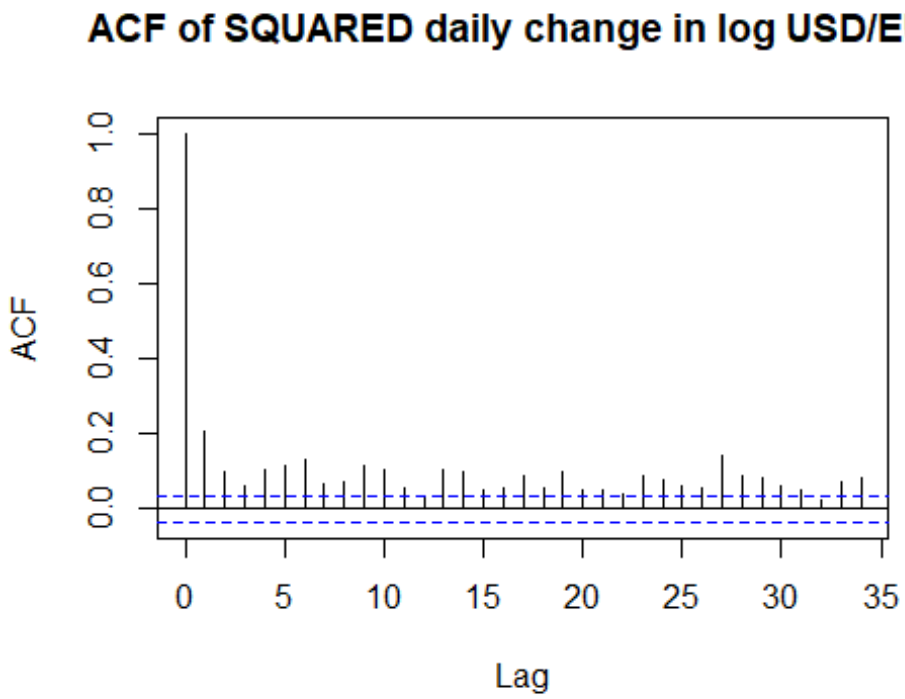
```
## Warning in adf.test(zFX): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  zFX
## Dickey-Fuller = -13.542, Lag order = 14, p-value = 0.01
## alternative hypothesis: stationary
```

```
x11()
acf(FX, main = "ACF of daily change in log USD/EUR")
```



```r
x11()
acf(FX^2, main = "ACF of SQUARED daily change in log USD/EUR")
```




```r
# Step 2

#test for ARCH as in Schwarz FOM Skript-------------------------
olstest = lm(FX ~ 1)
olstest

##
## Call:
## lm(formula = FX ~ 1)
##
## Coefficients:
## (Intercept)
##    0.004693

rt = residuals(olstest)
rt = rt[-1]
rt_1 = residuals(olstest)
rt_1 = rt_1[-length(rt_1)]

summary(lm(rt^2 ~ rt_1^2))

##
## Call:
## lm(formula = rt^2 ~ rt_1^2)
##
## Residuals:
```

```
##     Min      1Q  Median      3Q     Max
## -0.7208 -0.4097 -0.2790  0.0541 21.6005
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.45059    0.01733  25.995  < 2e-16 ***
## rt_1         0.10766    0.02583   4.169 3.15e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9629 on 3084 degrees of freedom
## Multiple R-squared:  0.005603,   Adjusted R-squared:  0.005281
## F-statistic: 17.38 on 1 and 3084 DF,  p-value: 3.148e-05
```

```
# Step 3

#=========================================================================
=
# NOW garch (danielsson, 2011) but notation as in fGarch Docu by
WurtzEtAlGarch.pdf
#=========================================================================
=

fit0 <- garchFit(~garch(1,1),cond.dist = "norm", data=FX,include.mean = T,
trace = FALSE)
summary(fit0) #see Gehrke 2022 393ff
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(1, 1), data = FX, cond.dist = "norm",
##     include.mean = T, trace = FALSE)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x000001c4b6f252a8>
##  [data = FX]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##       mu     omega    alpha1     beta1
## 0.016652  0.001671  0.031369  0.965223
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
```

```
## mu     0.0166516   0.0106787    1.559   0.1189
## omega  0.0016710   0.0007214    2.316   0.0205 *
## alpha1 0.0313690   0.0042462    7.388  1.5e-13 ***
## beta1  0.9652231   0.0045897  210.303  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2970.405    normalized:  -0.9622304
##
## Description:
##  Mon May  6 09:34:46 2024 by user: frank
##
##
## Standardised Residuals Tests:
##                                 Statistic p-Value
##  Jarque-Bera Test   R    Chi^2  125.179   0
##  Shapiro-Wilk Test  R    W      0.9941491 8.706726e-10
##  Ljung-Box Test     R    Q(10)  3.899319  0.9517746
##  Ljung-Box Test     R    Q(15)  13.63544  0.5533382
##  Ljung-Box Test     R    Q(20)  14.75284  0.7903712
##  Ljung-Box Test     R^2  Q(10)  5.428987  0.8607435
##  Ljung-Box Test     R^2  Q(15)  7.017233  0.9571698
##  Ljung-Box Test     R^2  Q(20)  9.182842  0.9806911
##  LM Arch Test       R    TR^2   5.790116  0.9262902
##
## Information Criterion Statistics:
##      AIC      BIC      SIC     HQIC
## 1.927052 1.934872 1.927049 1.929861
```

```
#hence set mue to zero
fit1 <- garchFit(~garch(1,1),cond.dist = "norm", data=FX,include.mean = F,
trace = FALSE)
fit1@fit$llh
```

```
## LogLikelihood
##      2971.618
```

```
fit1@fit$ics
```

```
##      AIC      BIC      SIC     HQIC
## 1.927190 1.933055 1.927188 1.929297
```

```
coef(fit1)
```

```
##       omega      alpha1       beta1
## 0.001682886 0.031009508 0.965544726
```

```
summary(fit1) #see Gehrke 2022 393ff
```

```
##
## Title:
##  GARCH Modelling
```

```
##
## Call:
##  garchFit(formula = ~garch(1, 1), data = FX, cond.dist = "norm",
##     include.mean = F, trace = FALSE)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x000001c4b38a7480>
##  [data = FX]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##     omega     alpha1      beta1
## 0.0016829  0.0310095  0.9655447
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## omega  0.0016829   0.0007237    2.326     0.02 *
## alpha1 0.0310095   0.0041855    7.409 1.27e-13 ***
## beta1  0.9655447   0.0045410  212.629  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2971.618    normalized:  -0.9626232
##
## Description:
##  Mon May  6 09:34:46 2024 by user: frank
##
##
## Standardised Residuals Tests:
##                                 Statistic p-Value
##  Jarque-Bera Test   R    Chi^2  127.6274  0
##  Shapiro-Wilk Test  R    W      0.9940898 7.408946e-10
##  Ljung-Box Test     R    Q(10)  3.939719  0.9500254
##  Ljung-Box Test     R    Q(15)  13.82707  0.5386831
##  Ljung-Box Test     R    Q(20)  15.02977  0.7747019
##  Ljung-Box Test     R^2  Q(10)  5.076129  0.8860348
##  Ljung-Box Test     R^2  Q(15)  6.72859   0.9647482
##  Ljung-Box Test     R^2  Q(20)  8.88158   0.9842387
##  LM Arch Test       R    TR^2   5.450219  0.9412289
##
## Information Criterion Statistics:
##      AIC      BIC      SIC     HQIC
## 1.927190 1.933055 1.927188 1.929297
```

```r
predict(fit1, n.ahead = 3)
```

```
##   meanForecast meanError standardDeviation
## 1            0 0.6400567         0.6400567
## 2            0 0.6402686         0.6402686
## 3            0 0.6404797         0.6404797
```

```r
# Step 4

fit2 <- garchFit(~garch(1,1),cond.dist = "std", data=FX,include.mean = FALSE,
            include.delta = FALSE,  trace = FALSE)
fit2@fit$llh
```

```
## LogLikelihood
##      2947.204
```

```r
fit2@fit$ics
```

```
##      AIC      BIC      SIC     HQIC
## 1.912021 1.919841 1.912018 1.914830
```

```r
coef(fit2)
```

```
##        omega       alpha1        beta1        shape
##  0.001978934  0.031576576  0.964447375 10.000000000
```

```r
summary(fit2)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(1, 1), data = FX, cond.dist = "std",
##     include.mean = FALSE, include.delta = FALSE, trace = FALSE)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x000001c4bcfd3e18>
##  [data = FX]
##
## Conditional Distribution:
##  std
##
## Coefficient(s):
##      omega      alpha1       beta1       shape
##  0.0019789   0.0315766   0.9644474  10.0000000
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
```

```
## omega  1.979e-03   8.599e-04    2.301   0.0214 *
## alpha1 3.158e-02   4.900e-03    6.444 1.16e-10 ***
## beta1  9.644e-01   5.352e-03  180.219  < 2e-16 ***
## shape  1.000e+01   1.464e+00    6.829 8.56e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2947.204    normalized:  -0.9547147
##
## Description:
##  Mon May  6 09:34:47 2024 by user: frank
##
##
## Standardised Residuals Tests:
##                                 Statistic p-Value
##  Jarque-Bera Test   R    Chi^2  128.8512  0
##  Shapiro-Wilk Test  R    W      0.9940576 6.790797e-10
##  Ljung-Box Test     R    Q(10)  3.976858  0.9483849
##  Ljung-Box Test     R    Q(15)  13.86805  0.5355592
##  Ljung-Box Test     R    Q(20)  15.07798  0.7719281
##  Ljung-Box Test     R^2  Q(10)  5.116729  0.8832451
##  Ljung-Box Test     R^2  Q(15)  6.824199  0.9623456
##  Ljung-Box Test     R^2  Q(20)  8.978839  0.9831511
##  LM Arch Test       R    TR^2   5.496303  0.9393194
##
## Information Criterion Statistics:
##      AIC      BIC      SIC     HQIC
## 1.912021 1.919841 1.912018 1.914830
```

```
# Step 5

#============================================================================
=
# NOW EGarch (Schröder)
#============================================================================
=

eGARCH_SET<-ugarchspec(variance.model =
list(model="eGARCH",garchOrder=c(1,1)),mean.model =
list(armaOrder=c(0,0),include.mean=FALSE))
fit3 <- ugarchfit(spec=eGARCH_SET,data =FX,solver="hybrid")
coef(fit3)

##        omega       alpha1        beta1       gamma1
## -0.002791764 -0.009082580  0.994859948  0.075070074

str(fit3)

## Formal class 'uGARCHfit' [package "rugarch"] with 2 slots
##   ..@ fit  :List of 27
##   .. ..$ hessian        : num [1:4, 1:4] 1315942 23186 -1253309 -41055
```

```
23186 ...
##   .. ..$ cvar           : num [1:4, 1:4] -1.23e-06 -1.29e-06 -2.81e-06
2.11e-05 -1.29e-06 ...
##   .. ..$ var            : num [1:3087] 0.45 0.425 0.422 0.451 0.435 ...
##   .. ..$ sigma          : num [1:3087] 0.671 0.652 0.65 0.671 0.659 ...
##   .. ..$ condH          : num NaN
##   .. ..$ z              : num [1:3087] 0.0126 -0.6126 -1.4615 0.3454 -
1.1753 ...
##   .. ..$ LLH            : num -2968
##   .. ..$ log.likelihoods: num [1:3087] 0.52 0.679 1.556 0.58 1.193 ...
##   .. ..$ residuals      : num [1:3087] 0.00848 -0.39944 -0.94974 0.23185 -
0.77493 ...
##   .. ..$ coef           : Named num [1:4] -0.00279 -0.00908 0.99486
0.07507
##   .. .. ..- attr(*, "names")= chr [1:4] "omega" "alpha1" "beta1" "gamma1"
##   .. ..$ robust.cvar    : num [1:4, 1:4] 8.61e-06 6.18e-06 8.72e-06 -
2.23e-05 6.18e-06 ...
##   .. ..$ A              : num [1:4, 1:4] 426.29 7.51 -406 -13.3 7.51 ...
##   .. ..$ B              : num [1:4, 1:4] 602.01 5.52 -499.74 -8.56 5.52
...
##   .. ..$ scores         : num [1:3087, 1:4] 0 0.312 -1.118 1.25 -0.723 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : NULL
##   .. .. .. ..$ : chr [1:4] "omega" "alpha1" "beta1" "gamma1"
##   .. ..$ se.coef        : num [1:4] 0.00111 0.00553 0.00194 0.0078
##   .. ..$ tval           : Named num [1:4] -2.51 -1.64 512.97 9.62
##   .. .. ..- attr(*, "names")= chr [1:4] "omega" "alpha1" "beta1" "gamma1"
##   .. ..$ matcoef        : num [1:4, 1:4] -0.00279 -0.00908 0.99486 0.07507
0.00111 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : chr [1:4] "omega" "alpha1" "beta1" "gamma1"
##   .. .. .. ..$ : chr [1:4] " Estimate" " Std. Error" " t value" "Pr(>|t|)"
##   .. ..$ robust.se.coef : num [1:4] 0.00293 0.00794 0.00323 0.01428
##   .. ..$ robust.tval    : Named num [1:4] -0.952 -1.143 307.739 5.259
##   .. .. ..- attr(*, "names")= chr [1:4] "omega" "alpha1" "beta1" "gamma1"
##   .. ..$ robust.matcoef : num [1:4, 1:4] -0.00279 -0.00908 0.99486 0.07507
0.00293 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : chr [1:4] "omega" "alpha1" "beta1" "gamma1"
##   .. .. .. ..$ : chr [1:4] " Estimate" " Std. Error" " t value" "Pr(>|t|)"
##   .. ..$ fitted.values  : num [1:3087] 0 0 0 0 0 0 0 0 0 0 ...
##   .. ..$ convergence    : num 0
##   .. ..$ kappa          : num 0.798
##   .. ..$ persistence    : num 0.995
##   .. ..$ timer          : 'difftime' num 0.677191019058228
##   .. .. ..- attr(*, "units")= chr "secs"
##   .. ..$ ipars          : num [1:19, 1:6] 0 0 0 0 0 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : chr [1:19] "mu" "ar" "ma" "arfima" ...
##   .. .. .. ..$ : chr [1:6] "Level" "Fixed" "Include" "Estimate" ...
##   .. ..$ solver         :List of 2
```

```
##   .. .. ..$ sol :List of 10
##   .. .. .. ..$ pars       : Named num [1:4] -0.00279 -0.00908 0.99486
0.07507
##   .. .. .. .. ..- attr(*, "names")= chr [1:4] "omega" "alpha1" "beta1"
"gamma1"
##   .. .. .. ..$ convergence: num 0
##   .. .. .. ..$ values     : num [1:5] 8672 2970 2968 2968 2968
##   .. .. .. ..$ lagrange   : num [1, 1] -2.55e-07
##   .. .. .. ..$ hessian    : num [1:5, 1:5] 0.39618 -0.00911 0.05471
0.90493 0.32476 ...
##   .. .. .. ..$ ineqx0     : Named num 1.01
##   .. .. .. .. ..- attr(*, "names")= chr ""
##   .. .. .. ..$ nfuneval   : num 255
##   .. .. .. ..$ outer.iter : num 4
##   .. .. .. ..$ elapsed    : 'difftime' num 0.653506994247437
##   .. .. .. .. ..- attr(*, "units")= chr "secs"
##   .. .. .. ..$ vscale     : num [1:6] 1 1 1 1 1 1
##   .. .. ..$ hess: NULL
##   ..@ model:List of 11
##   .. ..$ modelinc  : Named num [1:22] 0 0 0 0 0 0 1 1 1 1 ...
##   .. .. ..- attr(*, "names")= chr [1:22] "mu" "ar" "ma" "arfima" ...
##   .. ..$ modeldesc :List of 3
##   .. .. ..$ distribution: chr "norm"
##   .. .. ..$ distno      : int 1
##   .. .. ..$ vmodel      : chr "eGARCH"
##   .. ..$ modeldata :List of 4
##   .. .. ..$ T     : int 3087
##   .. .. ..$ data  : num [1:3087] 0.00848 -0.39944 -0.94974 0.23185 -
0.77493 ...
##   .. .. ..$ index : POSIXct[1:3087], format: "1970-01-02" ...
##   .. .. ..$ period: 'difftime' num 1
##   .. .. .. ..- attr(*, "units")= chr "days"
##   .. ..$ pars      : num [1:19, 1:6] 0 0 0 0 0 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : chr [1:19] "mu" "ar" "ma" "arfima" ...
##   .. .. .. ..$ : chr [1:6] "Level" "Fixed" "Include" "Estimate" ...
##   .. ..$ start.pars: NULL
##   .. ..$ fixed.pars: NULL
##   .. ..$ maxOrder  : num 1
##   .. ..$ pos.matrix: num [1:21, 1:3] 0 0 0 0 0 0 1 2 3 4 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : chr [1:21] "mu" "ar" "ma" "arfima" ...
##   .. .. .. ..$ : chr [1:3] "start" "stop" "include"
##   .. ..$ fmodel    : NULL
##   .. ..$ pidx      : num [1:19, 1:2] 1 2 3 4 5 6 7 8 9 10 ...
##   .. .. ..- attr(*, "dimnames")=List of 2
##   .. .. .. ..$ : chr [1:19] "mu" "ar" "ma" "arfima" ...
##   .. .. .. ..$ : chr [1:2] "begin" "end"
##   .. ..$ n.start   : num 0
```

```r
#eGARCH_VOLA <- as.numeric(ugarchforecast(fit3,
n.ahead=1)@forecast[["sigmaFor"]])*sqrt(252)

#compare
summary(fit1)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(1, 1), data = FX, cond.dist = "norm",
##     include.mean = F, trace = FALSE)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x000001c4b38a7480>
##  [data = FX]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##     omega     alpha1      beta1
## 0.0016829  0.0310095  0.9655447
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## omega  0.0016829   0.0007237    2.326     0.02 *
## alpha1 0.0310095   0.0041855    7.409 1.27e-13 ***
## beta1  0.9655447   0.0045410  212.629  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2971.618    normalized:  -0.9626232
##
## Description:
##  Mon May  6 09:34:46 2024 by user: frank
##
##
## Standardised Residuals Tests:
##                                 Statistic p-Value
##  Jarque-Bera Test   R    Chi^2  127.6274  0
##  Shapiro-Wilk Test  R    W      0.9940898 7.408946e-10
##  Ljung-Box Test     R    Q(10)  3.939719  0.9500254
##  Ljung-Box Test     R    Q(15)  13.82707  0.5386831
##  Ljung-Box Test     R    Q(20)  15.02977  0.7747019
```

```
##  Ljung-Box Test     R^2  Q(10)  5.076129  0.8860348
##  Ljung-Box Test     R^2  Q(15)  6.72859   0.9647482
##  Ljung-Box Test     R^2  Q(20)  8.88158   0.9842387
##  LM Arch Test       R    TR^2   5.450219  0.9412289
##
## Information Criterion Statistics:
##      AIC      BIC      SIC     HQIC
## 1.927190 1.933055 1.927188 1.929297
```

```
summary(fit2)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(1, 1), data = FX, cond.dist = "std",
##     include.mean = FALSE, include.delta = FALSE, trace = FALSE)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x000001c4bcfd3e18>
##  [data = FX]
##
## Conditional Distribution:
##  std
##
## Coefficient(s):
##      omega      alpha1       beta1       shape
##  0.0019789   0.0315766   0.9644474  10.0000000
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## omega  1.979e-03   8.599e-04    2.301   0.0214 *
## alpha1 3.158e-02   4.900e-03    6.444 1.16e-10 ***
## beta1  9.644e-01   5.352e-03  180.219  < 2e-16 ***
## shape  1.000e+01   1.464e+00    6.829 8.56e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2947.204    normalized:  -0.9547147
##
## Description:
##  Mon May  6 09:34:47 2024 by user: frank
##
##
## Standardised Residuals Tests:
##                                 Statistic p-Value
```

```
##  Jarque-Bera Test   R    Chi^2  128.8512  0
##  Shapiro-Wilk Test  R    W      0.9940576 6.790797e-10
##  Ljung-Box Test     R    Q(10)  3.976858  0.9483849
##  Ljung-Box Test     R    Q(15)  13.86805  0.5355592
##  Ljung-Box Test     R    Q(20)  15.07798  0.7719281
##  Ljung-Box Test     R^2  Q(10)  5.116729  0.8832451
##  Ljung-Box Test     R^2  Q(15)  6.824199  0.9623456
##  Ljung-Box Test     R^2  Q(20)  8.978839  0.9831511
##  LM Arch Test       R    TR^2   5.496303  0.9393194
##
## Information Criterion Statistics:
##      AIC      BIC      SIC     HQIC
## 1.912021 1.919841 1.912018 1.914830
```

```
fit3
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : eGARCH(1,1)
## Mean Model   : ARFIMA(0,0,0)
## Distribution : norm
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## omega  -0.002792    0.001111  -2.5139 0.011939
## alpha1 -0.009083    0.005528  -1.6431 0.100356
## beta1   0.994860    0.001939 512.9702 0.000000
## gamma1  0.075070    0.007804   9.6196 0.000000
##
## Robust Standard Errors:
##         Estimate  Std. Error   t value Pr(>|t|)
## omega  -0.002792    0.002934  -0.95151  0.34135
## alpha1 -0.009083    0.007945  -1.14319  0.25296
## beta1   0.994860    0.003233 307.73911  0.00000
## gamma1  0.075070    0.014276   5.25850  0.00000
##
## LogLikelihood : -2968.271
##
## Information Criteria
## ------------------------------------
##
## Akaike       1.9257
## Bayes        1.9335
## Shibata      1.9257
## Hannan-Quinn 1.9285
##
```

```
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.2281  0.6329
## Lag[2*(p+q)+(p+q)-1][2]    0.2365  0.8322
## Lag[4*(p+q)+(p+q)-1][5]    0.8380  0.8951
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                    0.04236  0.8369
## Lag[2*(p+q)+(p+q)-1][5]   0.81749  0.8994
## Lag[4*(p+q)+(p+q)-1][9]   2.31411  0.8644
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3] 0.0001589 0.500 2.000  0.9899
## ARCH Lag[5] 0.9891289 1.440 1.667  0.7362
## ARCH Lag[7] 2.1490910 2.315 1.543  0.6858
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  0.6241
## Individual Statistics:
## omega  0.09023
## alpha1 0.07129
## beta1  0.14551
## gamma1 0.32696
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.07 1.24 1.6
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value      prob sig
## Sign Bias            3.562 0.0003742 ***
## Negative Sign Bias   2.175 0.0296894  **
## Positive Sign Bias   3.136 0.0017310 ***
## Joint Effect        15.878 0.0012010 ***
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     36.17    0.0100647
## 2    30     46.09    0.0229840
```

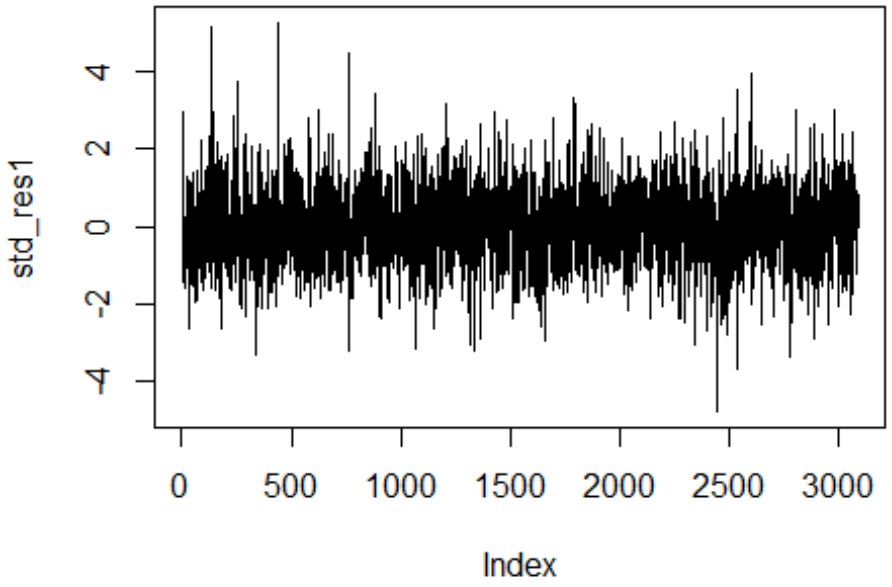
```
## 3    40     78.70    0.0001715
## 4    50     69.77    0.0271852
##
##
## Elapsed time : 0.677191

#=========================================================================
=
# LR Test
#=========================================================================
=

#LR Test
LR = fit1@fit$llh
LU = fit2@fit$llh

#test_stat as in Danielsson p 45 6 verbeek p191
#H0 restriction for shape is correct,
#L2 is Unrestricted cause free shape
LR_stats = 2*(LU-LR)
noofrestrictions = 1 #shape is set to infinity to make it normal
LR_crit = qchisq(0.99, noofrestrictions )

LR_crit

## [1] 6.634897

LR_stats

## LogLikelihood
##     -48.82741

#p-val, H0: restriction is true
round(2*pchisq(-abs(LR_stats),noofrestrictions ),5)

## LogLikelihood
##             0

# Step 6

#=========================================================================
=
# Apply GARCH student t
#=========================================================================
=

fit2

##
## Title:
##  GARCH Modelling
##
## Call:
```

```
##  garchFit(formula = ~garch(1, 1), data = FX, cond.dist = "std",
##     include.mean = FALSE, include.delta = FALSE, trace = FALSE)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x000001c4bcfd3e18>
##  [data = FX]
##
## Conditional Distribution:
##  std
##
## Coefficient(s):
##      omega      alpha1       beta1       shape
##  0.0019789   0.0315766   0.9644474  10.0000000
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## omega  1.979e-03   8.599e-04    2.301   0.0214 *
## alpha1 3.158e-02   4.900e-03    6.444 1.16e-10 ***
## beta1  9.644e-01   5.352e-03  180.219  < 2e-16 ***
## shape  1.000e+01   1.464e+00    6.829 8.56e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2947.204    normalized:  -0.9547147
##
## Description:
##  Mon May  6 09:34:47 2024 by user: frank
```

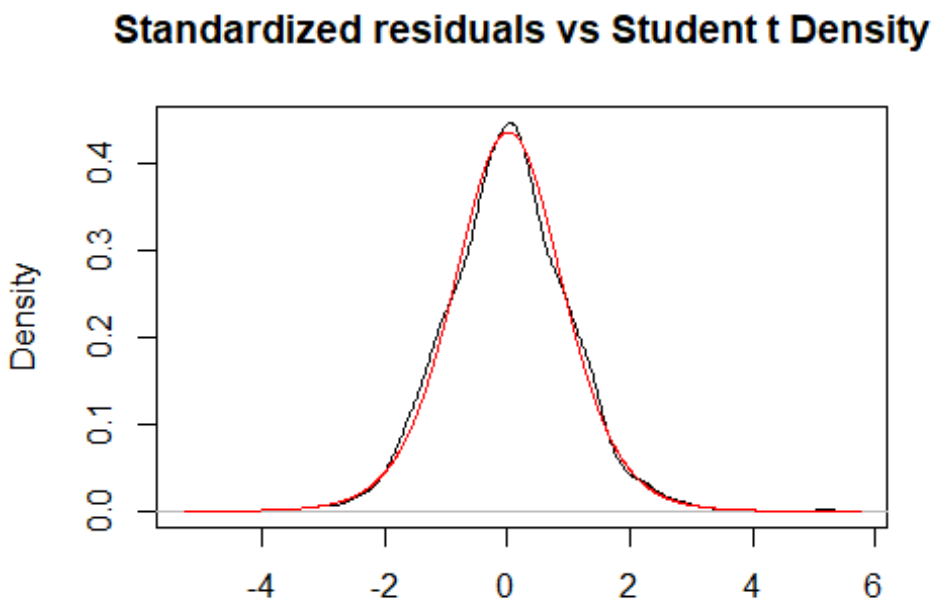
```r
Omega = coef(fit2)[1]
Alpha = coef(fit2)[2]
Beta = coef(fit2)[3]
Shape = coef(fit2)[4]

predict(fit2, n.ahead = 3)
```

```
##   meanForecast meanError standardDeviation
## 1            0 0.6394095         0.6394095
## 2            0 0.6396857         0.6396857
## 3            0 0.6399607         0.6399607
```

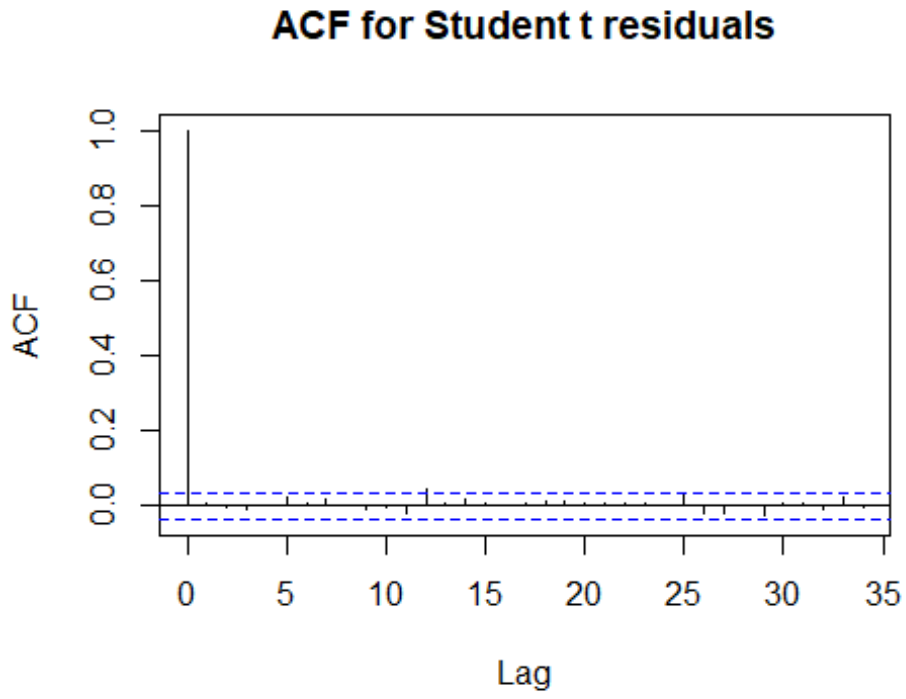
```r
std_res1 <- fit2@residuals/fit2@sigma.t #this standardizes the residuals to
mean = 0, s=1

x11()
plot(std_res1, type = 'l')
```



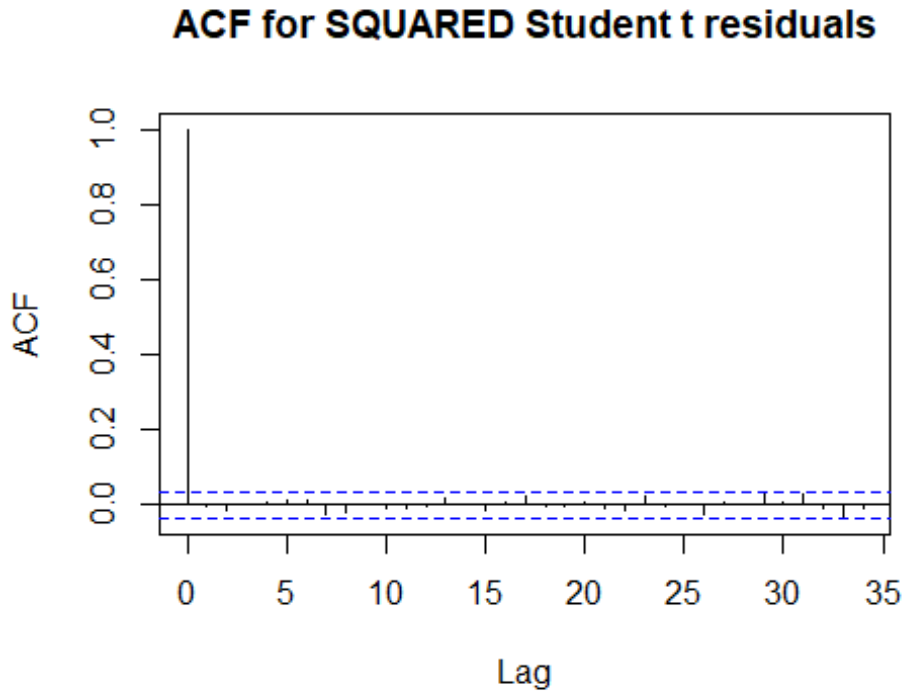

```
kernel_den = density(std_res1)
x11()
plot(kernel_den, main="Standardized residuals vs Student t Density", xlab = "
")
x <- seq(min(kernel_den$x),max(kernel_den$x),length = 1000)
std_den = dstd(x,mean = mean(std_res1),sd = sd(std_res1), nu = Shape)
lines(x,std_den,type="l",col="red")
```




```
#=============================================================================
=
# analyze the standardized residuals from the model
#=============================================================================
=
x11()
acf(std_res1, main="ACF for Student t residuals")
```

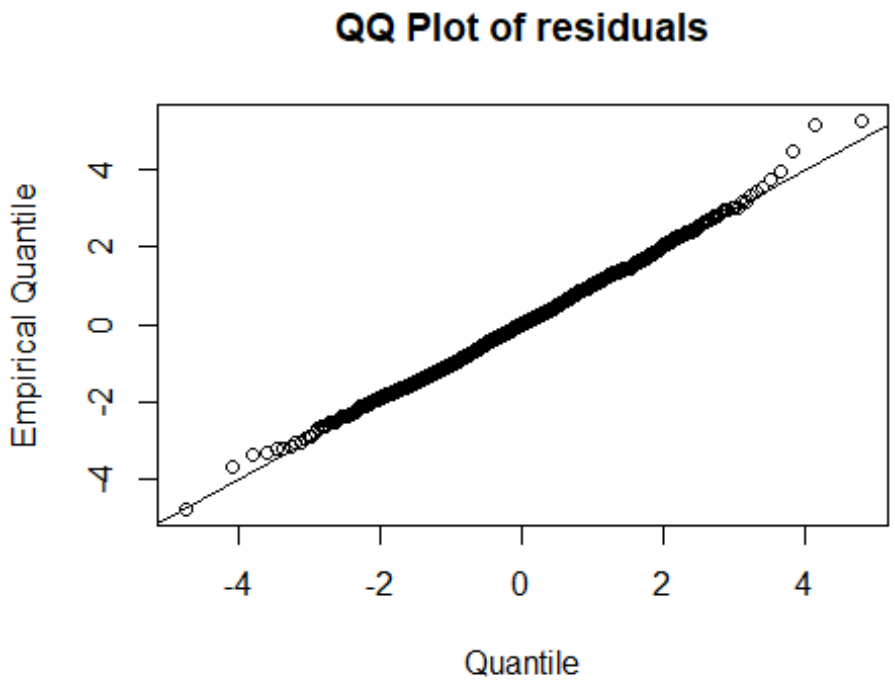



```
x11()
acf(std_res1^2, main="ACF for SQUARED Student t residuals")
```




```
#=========================================================================
=
```

```r
# do QQ & KS
#================================================================================
=
x11()
plot(qstd(ppoints(std_res1), mean = mean(std_res1),sd = sd(std_res1), nu =
Shape),
        sort(std_res1), main="QQ Plot of residuals", xlab = "Quantile", ylab
= "Empirical Quantile")
abline(a=0, b=1)
```




```r
#end
```